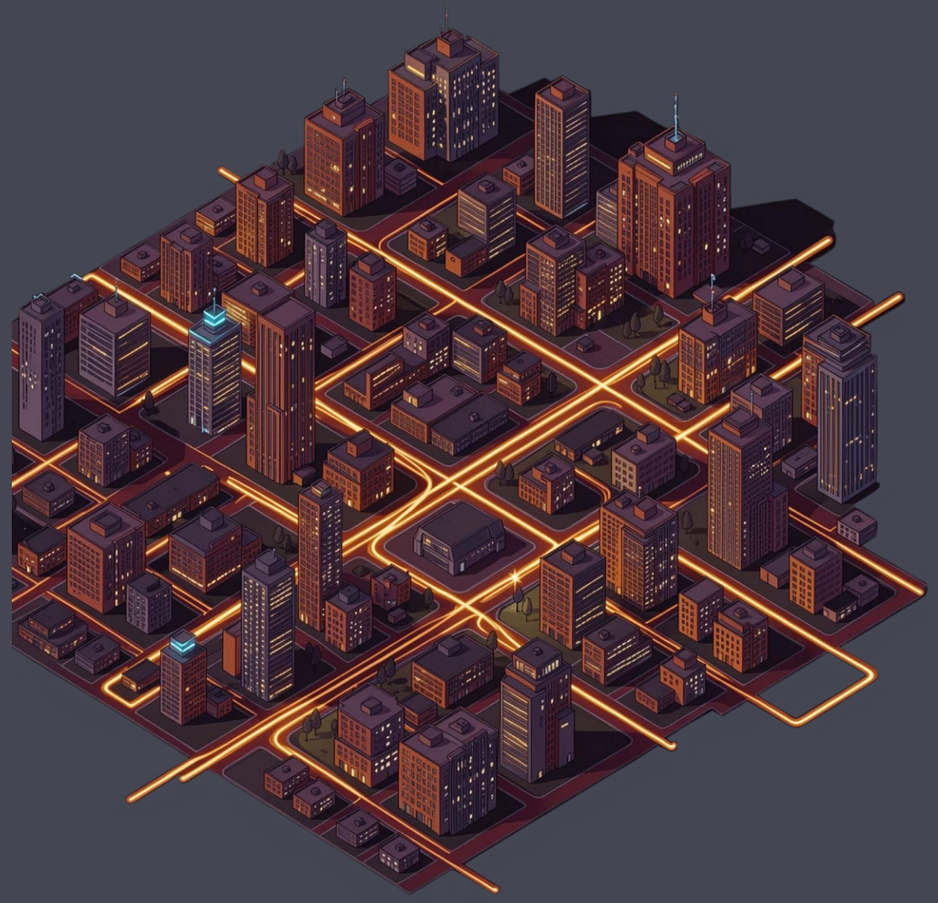


Heuristic Optimization for the Traveling Salesman Problem

Exact, Constructive, and Metaheuristic Approaches

ANJALI POLASANI

Track B: Optimization

- *Heuristic Design Rationale: Clear motivation for each algorithm choice*
- *Systematic vs. Stochastic Comparison: Quality-variance tradeoff analysis*
- *Scaling Analysis: Performance across problem sizes n=10 to n=200*

Table of Contents

- *Problem Statement*
- *Problem Formulation*
- *Design Approach*
- *Implementation*
- *Overall Best Algorithm & Key Findings*
- *Conclusion*
- *Future Work*

Problem Statement: The Traveling Salesman Problem

Imagine you're a delivery driver who needs to visit 50 cities and return home. What's the shortest route? This is the Traveling Salesman Problem (TSP).

TSP is one of the hardest problems in computer science. Even with powerful computers, finding the perfect answer takes too long. So we use smart shortcuts called heuristics to find good answers quickly.

Logistics

Finding the best delivery routes

Manufacturing

Organizing work tasks

Genomics

Organizing DNA data

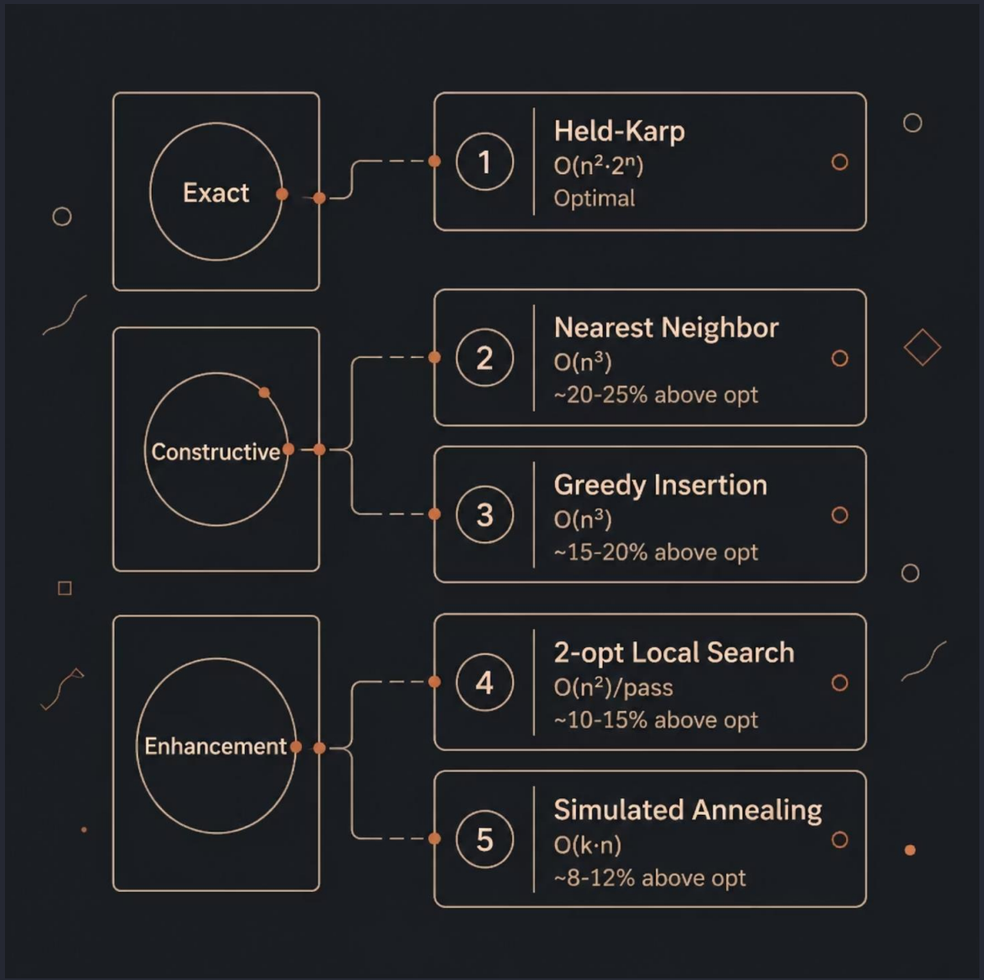
Astronomy

Scheduling observations

Problem Formulation

Given n cities and the distances between them, the Traveling Salesman Problem asks for the shortest route that visits each city exactly once and returns to the starting point. The goal is to minimize total travel distance while visiting all cities exactly once.

Design Approach: Three Algorithm Families



I implemented three algorithm families: Baseline (Held-Karp exact, Nearest Neighbor, Greedy Insertion) and Enhancement (2-opt Local Search, Simulated Annealing). I tested them on different problem sizes (10 to 200 cities) to see which works best.

Implementation Details: Baselines and Enhancements

Baselines

- *Held-Karp (Exact): I try every possible route in a smart way to find the best one. This gives the optimal answer, but only works for small problems.*
- *Nearest Neighbor (Constructive): I start at one city and keep going to the closest unvisited city. I test every start city and keep the best route.*
- *Greedy Insertion (Constructive): I build the route by inserting cities where they add the least extra distance.*

Enhancements

- *2-opt Local Search (Enhancement): I improve a route by swapping segments to remove crossings and shorten the trip. I repeat this until no easy improvement is left.*
- *Simulated Annealing (Metaheuristic): I start with a route and sometimes accept worse moves to escape local traps. Over time, it becomes more selective and keeps the best route found.*

Overall Best Algorithm & Key Findings

Best Overall Performance

- *Simulated Annealing was the best performer overall*
- *It gave the strongest results across the tested problem sizes*
- *It offered the best balance of solution quality and runtime*

SA achieved 5-10% improvement over Nearest Neighbor at n=50.

Project Highlights

- *Built and tested five algorithms from scratch*
- *Compared methods to see which ones worked best*
- *Tested across multiple problem sizes with repeated trials*

Conclusion

Heuristic algorithms can solve hard problems effectively and efficiently. Simulated Annealing performed best overall, and any structured approach outperformed random search by a wide margin. Structured heuristics consistently outperformed random search by approximately 77% at n=50.

Future Work

Future Directions

- *Try smarter tuning so the algorithm can adapt to each problem size.*
- *Explore a few stronger search moves to help it escape tougher local optima.*
- *Compare against other approaches and standard benchmarks for a broader evaluation.*

Thank You

Polasani Anjali

CS 57200: Heuristic Problem Solving

Purdue University Fort Wayne